



University of Cape Town

ECO5040W — Financial Software Engineering

Business and Technical Specification

XRPL-Based FX Remittance Platform using RLUSD

Annita Ngoma Karabo Tigedi Kerry-Lynn Whyte

Liltha Mzamo Nikola Milosavljevic

Group 3

7 September 2026

This specification describes **Baobab** (“Rooted here. Reaching there.”), **Group 3’s** XRPL remittance prototype, as implemented in the repository **kerryw33/FSE-Project**, which is the main source for the project. It began as Kerry-Lynn Whyte’s GitHub sketch, which established FastAPI, SQLite, Redis Streams, and Testnet Payments; the group’s patches were made on **Karabo22Tigedi/FSE-Project** and are merged back into it. This document is about *our* patched system, not the original sketch. Cash-in and cash-out remain simulated. Using a dollar stablecoin does not remove South African exchange-control, anti-money-laundering, or licensing duties (University of Cape Town, 2026a). One hundred and thirty-eight automated tests currently pass. The examiner-facing product is the Vite React UI in **frontend/** (Section 15), talking to FastAPI on port 8000.

1 Business problem

Cross-border remittances move household income to people who often collect cash at an agent. Western Union and MoneyGram are the familiar brands: the sender pays rand at an agent or online, the operator takes a fee, and the recipient collects cash on the other side after showing identification. Those operators are large, licensed money transmitters with agent networks in many countries. They are also expensive on small sends, and the price the customer sees is often a mix of a posted fee and a worse-than-mid-market exchange rate.

The World Bank’s Remittance Prices Worldwide (RPW) survey is the standard public comparison of corridor costs (World Bank, 2025). South Africa is a high-cost sending market on that survey. Rateweb’s 2026 reading of the 2025 Q1 RPW release reports Western Union and MoneyGram flat fees of about R214–R217 on a surveyed send of about R1,370 (Rateweb, 2026). That is a fee of roughly 15–16% of the send amount before any FX spread. For a family sending grocery money, that is a large bite.

Cost is not the only friction. The IMF’s diagnostic of the South Africa–Zimbabwe corridor describes cash-heavy, agent-based payouts, delays, and weak price transparency (International

Monetary Fund, 2025). FinMark Trust’s 2024 assessment of South Africa to the rest of SADC likewise finds that cash collection and agent processes still dominate, and that customers often cannot see the full cost until they are already at the counter (FinMark Trust, 2024). Western Union and MoneyGram solve a real problem—getting cash to someone without a bank account—but they do it with many intermediaries, posted-plus-spread pricing, and a payout model that is slow to digitise.

Stablecoins such as RLUSD (a US-dollar token issued on the XRP Ledger) can move value between wallets in seconds once both sides hold the token. That does not magically replace the cash-in door in Johannesburg or the cash-out door in Harare. The brief is explicit: a prototype may *simulate* those fiat legs (University of Cape Town, 2026a). Our platform therefore uses UCTUSD (Marc’s course IOU) as the current *settlement* asset between a confirmed rand payment and a custodial recipient wallet; the API slot remains `rlusd_amount` and can later point at official RLUSD via `.env` if funded. We do not claim that putting value on a ledger deletes the South African Reserve Bank, FICA, or a remittance licence.

What we do claim, as a design, is narrower. A quoted fee of a fixed rand amount plus a percentage, plus a published FX margin on the conversion rate, is easier to audit than an opaque corridor tariff. A 15-minute quote with an explicit cancel path stops abandoned drafts from consuming a sender’s daily limit. An on-chain Payment with a stored transaction hash gives the recipient something they can look up on a Testnet explorer. Those are the product bets this specification implements.

2 User journey

The journey follows the brief (University of Cape Town, 2026a) and is implemented end-to-end on the API and on the web UI in Section 15.

1. A customer registers (`POST /auth/register`) and logs in (`POST /auth/login`). The same customer account can send or receive; role is `customer` or `admin`, not two person types.
2. The sender submits mock KYC (`POST /kyc`). An admin approves or rejects it. Unapproved senders cannot create quotes.
3. The sender adds a beneficiary (`POST /beneficiaries`). If mobile or email already matches a registered user, the record links immediately; if the recipient registers later, linking happens then.
4. The sender posts a ZAR amount (`POST /remittances`). The API fetches USD/ZAR, applies the fee model, checks daily and monthly limits, and returns a quote with `tracking_ref` (MG plus ten digits) and `expires_at` (15 minutes).
5. The sender may cancel an unexpired quote (`POST /remittances/{id}/cancel`) or let it expire. Cancelled and expired quotes do not consume limits.
6. The sender initiates simulated cash-in (agent cash, bank transfer, or card). An admin confirms receipt. Settlement is not queued before that confirmation.
7. A Redis Streams worker submits an XRPL Testnet Payment from the platform treasury to the recipient’s custodial account, stores the hash, and credits the internal spendable balance.
8. The recipient opens `GET /wallet/me`, sees spendable UCTUSD, incoming transfers with hashes, and cash-out history.

9. The recipient requests a cash-out to USD or ZAR. Fiat payout is still simulated, but complete submits a Payment of the reserved UCTUSD from the recipient custodial account to the official UCTUSD issuer (burn/redeem). `on_chain_balance` excludes completed cash-outs. Failed cash-outs refund spendable balance.

Anyone with a tracking reference who is the sender, the linked recipient, or an admin can open `/app/track/{ref}` or call `GET /remittances/track/{ref}`.

3 Functional requirements

The requirements below are the group's Functional Requirements v2. Each traces to a stage of the core user journey (Registration → KYC → Beneficiary → Quote → Cash-in → Queue/Settlement → Wallet → Cash-out) or to a supporting admin function. The third column names where each one is implemented, checked against the code. Every requirement is met in full except FR-04, which is met in part by design (Section 13). Interactive OpenAPI lives at `/docs`; `GET /health` is unauthenticated.

3.1 Registration and authentication

ID	Requirement	Implemented in
FR-01	Register an account with full name, email address, mobile number, and password.	<code>POST /auth/register</code> ; UI <code>/register</code> . Mobile numbers must be 8–15 digits with an optional leading +; passwords are bcrypt-hashed.
FR-02	Log in with registered credentials.	<code>POST /auth/login</code> returns a bearer token; only its SHA-256 hash is stored.
FR-02a	Log out, terminating the session.	<code>POST /auth/logout</code> sets <code>revoked_at</code> on the session.
FR-03	Email address and mobile number are unique across accounts.	Unique constraints on <code>users.email</code> and <code>users.mobile_number</code> ; a duplicate registration returns 409.
FR-04	Distinguish sender, recipient, and administrator roles.	<i>Partial, by design.</i> Stored roles are <code>customer</code> and <code>admin</code> ; sender and recipient are what a customer does on a given transfer (Section 13). Admins come only from <code>scripts/create_admin.py</code> or the <code>ADMIN_EMAIL/ADMIN_PASSWORD</code> start-up setting.
FR-04a	View and update basic profile information.	<code>GET/PATCH /users/me</code> ; UI <code>/app/profile</code> .

3.2 KYC

ID	Requirement	Implemented in
FR-05	Collect full name, date of birth, nationality, identification number, residential address, mobile number, email address, and source of funds.	<code>POST /kyc</code> stores all eight fields. The ID number (6–20 characters) is encrypted with the KYC Fernet key.
FR-06	Submit a completed KYC application for review.	<code>POST /kyc</code> sets status <code>pending</code> ; resubmission overwrites the application until it is approved.
FR-07	Administrator can view a submitted application.	<code>GET /kyc</code> , <code>GET /kyc/{id}</code> ; UI <code>/admin/kyc</code> .
FR-08	Administrator can approve or reject an application.	<code>POST /kyc/{id}/approve</code> or <code>/reject</code> (with a reason); the UI confirms the decision in a dialog naming the applicant.

ID	Requirement	Implemented in
FR-08a	User can view their own KYC status.	<code>GET /kyc/me/status</code> , <code>GET /kyc/me</code> ; shown on the sender home screen.
FR-09	No remittance without approved KYC.	<code>POST /remittances</code> requires approved KYC; the UI shows “KYC pending” instead of the send form.
FR-09a	No cash-out without approved KYC.	<code>POST /cash-outs</code> requires approved KYC. A recipient can still receive and hold UCTUSD without it.

The assumption attached to FR-09a is adopted as stated: senders need approved KYC before sending and recipients before cashing out, but receiving and holding needs none.

3.3 Beneficiary management

ID	Requirement	Implemented in
FR-10	Approved sender adds a beneficiary with full name, mobile number or email, country, payout currency, and relationship.	<code>POST /beneficiaries</code> , approved senders only; all five fields stored. UI <code>/app/beneficiaries</code> .
FR-11	Require at least a mobile number or an email address.	Request validation rejects a beneficiary with neither (422).
FR-12	Sender can list their beneficiaries.	<code>GET /beneficiaries</code> , each with a Linked or Unlinked badge in the UI.
FR-12a	Link a beneficiary to a recipient account by matching mobile number or email.	Auto-match when the beneficiary is added, and again whenever a new account registers (<code>services/beneficiary_linking.py</code>).
FR-12b	If no account matches, leave the beneficiary unlinked; no wallet and no settlement until a matching account registers.	The record stays unlinked. If cash-in is confirmed anyway, the worker refuses to settle (“Beneficiary is not linked to a registered recipient account”), creates no wallet, credits nothing, and marks the transfer <code>settlement_failed</code> ; an administrator retries once the recipient registers and is linked.
FR-12c	Provision a custodial wallet only once the beneficiary is linked to a registered recipient account.	The recipient’s custodial Testnet account is created at first settlement, and only for the linked user.

The linking mechanism left open in the requirements is decided as auto-match on an existing or later-registered account; there is no invite-to-register flow.

3.4 Quote, fees, and limits

ID	Requirement	Implemented in
FR-13	Determine the USD/ZAR rate when the transaction is created (public API, mock, or rate table).	<code>open.er-api.com</code> at quote creation, cached for five minutes, with a <code>.env</code> fallback rate (Section 5). The rate used is stored on the quote.

ID	Requirement	Implemented in
FR-14	Quote shows ZAR amount, rate, transaction fee, FX margin, RLUSD amount, cash-out fee, and estimated payout.	POST <code>/remittances</code> returns <code>zar_amount</code> , <code>exchange_rate</code> , <code>transaction_fee_zar</code> , <code>fx_margin_percentage</code> , <code>rlusd_amount</code> , <code>estimated_cash_out_fee</code> , and <code>estimated_recipient_payout</code> , plus <code>tracking_ref</code> and <code>expires_at</code> .
FR-15	Fees use a configurable fixed fee, percentage fee, FX margin, and cash-out fee.	One <code>FeeConfig</code> row (Section 4).
FR-15a	Administrator can view and update the fee parameters.	GET/PUT <code>/admin/fee-config</code> ; UI <code>/admin/config</code> .
FR-16	Check the amount against daily and monthly limits before proceeding.	Checked before the quote is inserted; cancelled and expired quotes do not count (Section 6).
FR-16a	Sender can view limits and remaining allowance.	GET <code>/limits/me</code> returns limits, usage, and remaining amounts for today and this month.
FR-16b	Administrator can view and update limit tiers.	GET <code>/admin/limit-tiers</code> , PUT <code>/admin/limit-tiers/{tier_key}</code> ; UI <code>/admin/config</code> .
FR-17	Reject a request that would exceed a limit, and say why.	HTTP 422 naming the limit, e.g. “This remittance would exceed your daily limit of ... ZAR”.

3.5 Simulated cash-in

ID	Requirement	Implemented in
FR-18	Sender initiates a simulated ZAR payment by a supported method.	POST <code>/remittances/{id}/cash-in</code> with <code>agent_cash</code> , <code>bank_transfer</code> , or <code>card</code> ; only for an unexpired, uncanceled quote.
FR-19	Administrator or mock service confirms the payment.	POST <code>/remittances/{id}/confirm-cash-in</code> ; UI <code>/admin/cash-in</code> .
FR-20	No settlement until cash-in is confirmed.	Settlement is enqueued only inside <code>confirm-cash-in</code> ; no Payment is submitted before it.

3.6 Queue and settlement

ID	Requirement	Implemented in
FR-21	Queue a settlement message once cash-in is confirmed.	<code>Confirm-cash-in</code> writes a <code>SettlementMessage</code> row (outbox) and adds it to the Redis Stream <code>settlement_messages</code> .
FR-22	A worker consumes messages and submits the transfer to XRPL Testnet.	<code>scripts/run_settlement_worker.py</code> or POST <code>/admin/settlement/run</code> reads the stream as a consumer group and submits a UCTUSD Payment with <code>xrpl-py</code> .
FR-23	Record each outcome, with the transaction hash on success.	Hash or failure reason stored on the remittance and the message; GET <code>/admin/settlement</code> ; UI <code>/admin/settlement</code> .
FR-24	A failed transaction does not credit the recipient.	Status becomes <code>settlement_failed</code> with no credit; covered by <code>tests/test_settlement.py</code> .

ID	Requirement	Implemented in
FR-25	A duplicate message cannot credit twice.	One message per remittance (unique constraint); if a hash is already stored the worker marks the job complete without paying or crediting; retrying a completed message returns 409.
FR-26	Update the wallet balance once settlement is confirmed on-ledger.	The worker waits for ledger validation, then stores the hash, credits the balance, and sets settled in one database commit.

3.7 Recipient wallet and sender history

ID	Requirement	Implemented in
FR-27	Each recipient has a custodial web wallet.	A custodial Testnet account per recipient, secret encrypted with its own Fernet key; <code>GET /wallet/me</code> ; UI <code>/app/wallet</code> .
FR-28	Wallet shows balance, incoming transfers, cash-outs, status, date, and transaction hash.	Spendable and on-chain balances; incoming transfers with amount, status, dates, and hash; cash-outs with amount, currency, status, dates, and burn hash.
FR-28a	Sender can view status and history of sent remittances.	<code>GET /remittances/me</code> and <code>GET /remittances/track/{ref}</code> ; UI <code>/app/history</code> and <code>/app/track/{ref}</code> show the quote breakdown, a status timeline, the hash, and a scannable QR code for the track page.

3.8 Simulated cash-out

ID	Requirement	Implemented in
FR-29	Establish a TrustLine to the issuer for any wallet holding the token.	TrustSet for the platform treasury (<code>scripts/setup_platform_wallet.py</code> or <code>POST /admin/platform-wallet/trustline</code>) and for each recipient account before its first payment.
FR-30	Recipient requests a cash-out to USD or local fiat.	<code>POST /cash-outs</code> to USD or ZAR; the amount is reserved from the spendable balance at once.
FR-31	Payout uses the applicable rate, less the cash-out fee.	Fee taken from the UCTUSD amount; USD pays out 1:1, ZAR at the live or fallback USD/ZAR rate without the FX margin.
FR-32	Track cash-outs as requested, approved, completed, or failed.	requested → approved → completed , or failed from requested or approved. Complete burns the reserved UCTUSD by paying the issuer; fail refunds the spendable balance.

3.9 Administration

ID	Requirement	Implemented in
FR-33	Admin interface for KYC applications.	UI <code>/admin/kyc</code> (FR-07, FR-08).
FR-34	Admin interface for confirming cash-in.	UI <code>/admin/cash-in</code> (FR-19).

ID	Requirement	Implemented in
FR-35	Admin interface for actioning cash-outs.	UI <code>/admin/cash-out</code> : POST <code>/cash-outs/{id}/approve</code> , <code>/complete</code> , <code>/fail</code> .

FR-15a and FR-16b are also administration functions; they are listed with the fees and limits they configure. Beyond the requirements, the system adds a 15-minute quote expiry with cancel (POST `/remittances/{id}/cancel`), MG tracking references, admin settlement retry, and a platform-wallet screen reporting treasury TrustLine and liquidity.

4 Fee model

The platform earns in three places: a transaction fee when the sender pays in rand, a margin on the exchange rate, and a fee when the recipient cashes out. The four numbers behind these charges are kept in a single settings record, `FeeConfig`. An administrator can change them on the Config screen (`/admin/config`, which calls PUT `/admin/fee-config`), and the next quote uses the new values, with no code change or restart. Only the current values are kept, not a history. Quotes already issued are unaffected, because each quote stores the fee and rate it was priced at.

The system starts with a fixed fee of R25.00, a percentage fee of 1% of the amount sent, an FX margin of 2%, and a cash-out fee of 1.5%. Percentages are stored as fractions, so 1% is stored as 0.0100.

The process for calculating a quote is as follows.

The *transaction fee* is the *fixed fee* plus a percentage of the amount sent (`zar_amount`), and it comes off the rand amount first. The remainder (*net_zar*) is then converted to dollars at the *effective rate* (the mid-market rate, marked up by the FX margin):

```
transaction_fee = fixed_fee + zar_amount * percentage
effective_rate  = midmarket * (1 + fx_margin)
net_zar         = zar_amount - transaction_fee
rlusd           = net_zar / effective_rate
```

Because the margin raises the number of rand needed for each dollar, the sender receives fewer dollars than the mid-market rate would give. That difference is the platform's exchange-rate income, and the quote shows it rather than hiding it. The token is treated as worth exactly one US dollar. Fees are rounded to the nearest cent and token amounts to six decimal places, with halves rounded up. If the fee is as large as the amount sent, nothing is left to convert, so the quote is refused (HTTP 422).

The cash-out fee on the quote is only an estimate, because the recipient may cash out later, in part, or in a different currency. When they do, the cash-out fee is taken from the token amount at the rate set at that time. A USD cash-out then pays one dollar per token; a ZAR cash-out converts at the current USD/ZAR rate, or the fallback rate, with no FX margin added.

4.1 Worked example

Using seeded defaults and the configured fallback mid-market rate of 18.50 (the live API may differ; the algebra does not). For a R1,370.00 send, comparable to the RPW send size discussed by Ratweb

(2026):

$$\text{transaction_fee} = 25.00 + 1370.00 \times 0.01 = 38.70$$

$$\text{effective_rate} = 18.50 \times 1.02 = 18.87$$

$$\text{net_zar} = 1370.00 - 38.70 = 1331.30$$

$$\text{rlusd} = 1331.30 / 18.87 = 70.551139$$

Estimated cash-out fee = $70.551139 \times 0.015 = 1.058267$ RLUSD; estimated recipient payout = 69.492872 RLUSD. The transaction fee (R38.70) is far below the R214–R217 Western Union / MoneyGram flat fees on that send size (Rateweb, 2026). That comparison is indicative only: our cash legs are simulated, we are not an agent network, and a live mid-market rate plus a 2% margin is not a licensed tariff.

A send whose fee consumes the principal is rejected. At the same defaults, R25.00 yields a R25.25 fee and a negative net, so the API returns 422.

Table 10 is the seeded `FeeConfig` an admin can edit at PUT `/admin/fee-config`. There is no fee-history table: the live row is what the next quote uses.

Table 10: Seeded fee parameters (fractions, not whole percents).

Field	Meaning	Default
<code>fixed_fee_zar</code>	Rand added to every send	25.00
<code>percentage_fee</code>	Share of send ZAR	0.0100
<code>fx_margin_percentage</code>	Added to mid-market rate	0.0200
<code>cash_out_fee_percentage</code>	Share of RLUSD at cash-out	0.0150

5 Exchange rates

The brief allows the exchange rate to come from a public API, a mock service, or a configured rate table (University of Cape Town, 2026a). We use a public API, with a configured rate as the backup.

When a quote is created, the system requests the latest US dollar rates from a free public service (`open.er-api.com`) and takes the rand rate from the reply. It keeps that rate for five minutes and reuses it for every quote made in that time, so that a burst of users does not flood a free service with requests. The first quote after the five minutes are up fetches a fresh rate. The five-minute period is set by `EXCHANGE_RATE_CACHE_SECONDS`.

If the request fails for any reason (no network, a timeout, an error response, or a reply with no rand rate), the quote is not refused. Instead the system uses a fixed backup rate from the settings file, `USD_ZAR_RATE` (18.50 unless changed), and keeps it for five minutes before trying the live service again. The backup rate cannot be changed from the admin screens; it is changed by editing the settings file and restarting the server, and it only matters while the live service is down.

The rate used, with the FX margin applied, is stored on each quote, so the sender sees the rate they were actually charged. The automated tests never contact the live service; they substitute a fixed or simulated rate so that results are repeatable.

6 Remittance limits

Tiers are derived from KYC, not stored on the user: approved $\rightarrow$ **verified**, otherwise **unverified**. Seeded caps match the brief's example: unverified R0 / R0 per day and month; verified R3,000 / R25,000. Admins may change caps via `PUT /admin/limit-tiers/{tier_key}`. Unverified is not reachable through `POST /remittances` because that route requires approved KYC; the zero caps still exist so the example table is configurable.

Usage sums the sender's ZAR since the start of the UTC day or calendar month, with two exclusions that the original sketch did not have: **cancelled** never counts; **quoted** rows with **expires_at** already in the past never count. Everything else counts: quotes that have not yet expired, and every status from cash-in onwards, including **settlement_failed**. This is deliberate. A failed on-chain payment still follows a confirmed cash-in, so the sender's rand was received and the amount stays counted. An administrator can retry the settlement, but nothing releases the amount early; it stops counting only when the day or month ends.

Table 11: Remittance statuses on one evolving row.

Status	How we get there	Counts to limit?
quoted	<code>POST /remittances</code>	Yes, until TTL
cancelled	sender cancel, still unexpired	No
quoted past TTL	clock; cash-in 409	No
cash_in_pending	sender cash-in	Yes
cash_in_confirmed	admin confirm	Yes
settlement_queued	enqueue after confirm	Yes
settled	worker Payment + credit	Yes
settlement_failed	worker error; no credit	Yes

7 Architecture

Figure 1 is the runtime. The Vite React app on 5173 (or 5174 if 5173 is busy) is the marked web UI. FastAPI serves JSON. SQLAlchemy talks to SQLite through Alembic revisions 0001_initial, 0002_quote_session, and 0003_cash_out_burn. Redis is transport for settlement, not the source of truth. The settlement worker (the same function as `POST /admin/settlement/run` and `python -m scripts.run_settlement_worker`) signs Testnet Payments with `xrpl-py`. CORS allows the local Vite and API origins, including loopback.

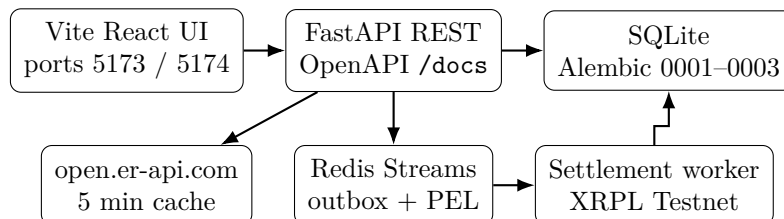


Figure 1: Main components. Redis carries settlement messages; durable status lives in SQLite.

Figure 2 is the money path the brief asks for.

Wallet design is a hybrid of the brief's two options (University of Cape Town, 2026a,b). One **PlatformWallet** holds scarce IOU liquidity. Each recipient still receives a real custodial Testnet

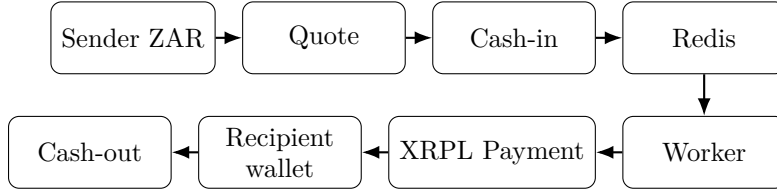


Figure 2: Settlement path: confirmed rand in, queued Payment, custodial UCTUSD, simulated fiat out.

account, generated lazily on first settlement, so FR-style “individually attributable Payment + hash” is a genuine ledger transaction rather than a book entry that pretends to be one. Address and encrypted secret are committed *before* TrustSet. If an address exists but the trust line is not yet established, the worker reuses the stored secret; it does not mint a second wallet.

XRP is the only asset an XRPL account holds natively; UCTUSD and RLUSD are issued tokens (IOUs). A holder can accept an IOU only after a TrustSet opens a trust line to the issuer, which reserves a little XRP (locked, not spent) (University of Cape Town, 2026b). Rippling lets the same issued token move between holders of that issuer without an explicit hop back through the issuer: Default Ripple is normally on for the issuer and off for ordinary holders. We do not implement rippling ourselves; settlement is a direct treasury-to-recipient Payment after TrustSet.

8 Cash-in, UCTUSD settlement, and cash-out

8.1 Cash-in

POST /remittances/{id}/cash-in is allowed only on an unexpired, non-cancelled `quoted` remittance. The sender chooses `agent_cash`, `bank_transfer`, or `card`. Status becomes `cash_in_pending`. Admin `confirm-cash-in` sets `cash_in_confirmed` and calls `enqueue_settlement`. No Payment is submitted at this step. The cash movement itself is simulated: there is no card acquirer, no agent till, and no bank confirmation webhook.

8.2 UCTUSD settlement

Enqueue inserts at most one `SettlementMessage` per remittance (unique constraint) and `XADDs` to stream `settlement_messages`. The Redis Stream ID is stored on `stream_entry_id`. If the database row exists but the stream ID is missing or the entry is gone, a later enqueue *republishes*—that covers “DB commit succeeded, Redis `XADD` failed”. Redis is transport; the database row is source of truth.

The worker first reclaims its pending-entries list (`XREADGROUP` id 0), then idle pending entries (`XAUTOCLAIM` on Redis 6.2+, else `XPENDING+XCLAIM`; local Memurai 5.0.14 has no `XAUTOCLAIM`), then new > entries. Entries are acknowledged only after the message is `completed` or `failed`. A crash mid-flight leaves the entry in the PEL for reclaim.

Before submitting a Payment, if `xrpl_settlement_tx_hash` is already set, the worker marks completed and **does not pay or credit again**. If no hash is stored, the worker first searches the treasury’s validated ledger history for a successful Payment whose memo carries this remittance id, and adopts that hash instead of paying again. This covers a crash after the ledger accepted a Payment but before the database recorded it; if the lookup itself fails, the worker does not pay. On the success path the Payment memo carries the remittance id; the hash, recipient credit, `settled`,

and `completed` are one database commit. Admin retry accepts `failed`, `processing`, and `pending` (reset to `pending`, republish). `completed` returns 409.

8.3 Cash-out

The recipient must have approved KYC to cash out but may hold UCTUSD without it. Spendable `RecipientWallet.balance` is debited atomically at request time (`UPDATE ...WHERE balance >=:amt`). `GET /wallet/me` reports `balance_rlusd` and `spendable_balance` as that spendable figure (the slot is currently UCTUSD; official RLUSD later via `.env` if funded). `on_chain_balance` is spendable plus cash-out amounts in `requested` or `approved` only; it excludes `completed` cash-outs. Fiat payout is still simulated, but complete submits a Payment of the reserved UCTUSD from the recipient custodial account to the official UCTUSD issuer (burn/redeem). After a request, spendable drops while on-chain stays higher by the reserved amount until complete, when the on-chain figure drops too. A failed cash-out refunds spendable.

9 Database

Runtime schema is Alembic, not `create_all` alone. FastAPI lifespan runs `alembic upgrade head` then seeds default fees and limit tiers. Tests still use in-memory `create_all`. Figure 3 is the entity–relationship view; the list below is the column notes.

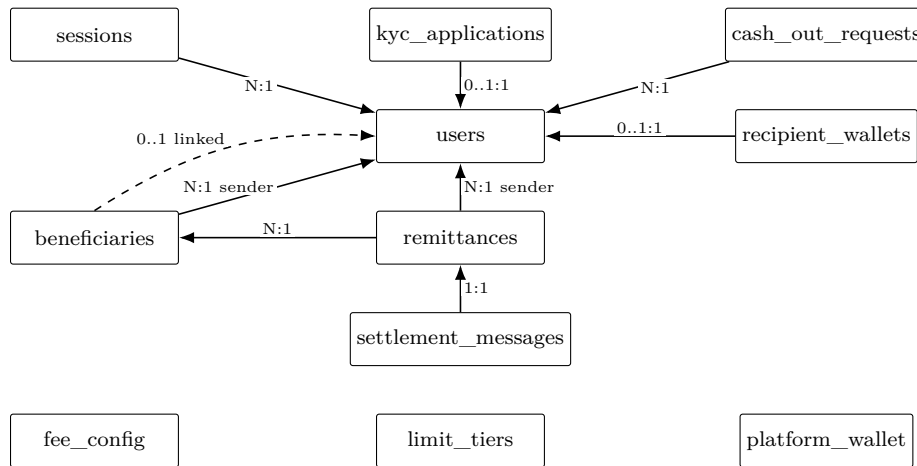


Figure 3: Entity–relationship view of the Alembic schema. Cardinalities are child:parent. The dashed edge is optional `linked_user_id`. Cash-out is not foreign-keyed to a remittance: payouts draw aggregated spendable UCTUSD. `fee_config` and `platform_wallet` are singletons; `limit_tiers` is the unverified/verified lookup.

- `users` — name, unique email and mobile, bcrypt `password_hash`, role.
- `sessions` — `token_hash` (SHA-256 hex, unique), expiry, `revoked_at`. No plaintext token column.
- `kyc_applications` — one row per user; identification number encrypted.
- `beneficiaries` — payout profile; optional `linked_user_id`.
- `remittances` — full lifecycle on one row: amounts, fees, status (including `cancelled`), `tracking_ref`, `expires_at`, cash-in fields, XRPL hash.

- `settlement_messages` — one per remittance; status; `stream_entry_id`; attempts; failure reason.
- `recipient_wallets` — custodial address, encrypted secret, trust-line flag, spendable `balance`. No `on_chain_balance` column; that figure is computed on read.
- `platform_wallet` — singleton treasury address and encrypted secret.
- `cash_out_requests` — RLUSD amount, fiat, fee, payout, status, admin action stamps.
- `fee_config, limit_tiers` — singleton fees; unverified/verified caps.

Encrypted columns are stored as `VARCHAR(500)` (Fernet ciphertext). Existing SQLite files created only with `create_all` have no `alembic_version` table and will fail upgrade; demo databases should start empty and migrate. A database left half-migrated by an interrupted upgrade fails the same way, because Alembic treats SQLite DDL as non-transactional: back it up and reconcile it by hand, or start again.

10 API overview

The API is JSON over HTTP with Bearer sessions. FastAPI emits Swagger UI at <http://127.0.0.1:8000/docs> and the OpenAPI schema at `/openapi.json`. Route prefixes: `auth`, `users`, `kyc`, `beneficiaries`, `remittances`, `limits`, `wallet`, `cash-outs`, and `admin` (fees, limits, settlement, platform wallet). CORS defaults allow Vite on 5173 and 5174 (`localhost` and `127.0.0.1`), local React on 3000, and the API host on 8000, overridable with `CORS_ORIGINS` as a JSON list. Credentials are allowed. The SPA stores the bearer token in `sessionStorage` and sends it as `Authorization: Bearer`.

Authz in short: customers act on their own resources; KYC-approved customers create quotes, beneficiaries, cash-in, and cash-outs; admins review KYC, confirm cash-in, run and retry settlement, action cash-outs, and edit fees and tiers. Tracking is visible to the sender, the linked recipient, or an admin (403 otherwise; 404 if the ref does not exist).

11 Security

Passwords are bcrypt via `passlib` and never stored in plaintext. Login returns a urlsafe bearer token once; the database stores only `SHA-256` of that token. Logout sets `revoked_at`. Opaque sessions were chosen over JWT because this is a single FastAPI process: logout must actually revoke access, which a signed JWT cannot do without a server-side denylist that looks like the sessions table anyway.

Two Fernet keys are required: `KYC_ENCRYPTION_KEY` for the identification number, `XRPL_KEY_ENCRYPTION_KEY` for platform and recipient XRPL secrets. A leak of one category does not decrypt the other. Keys live in environment variables, not in the database, not in API responses, not in application logs, and not in git (`.env` is local; `.env.example` has placeholders). XRPL secrets load from the database as ciphertext and are decrypted only immediately before signing, and the wallet and platform-wallet response models have no secret field, so a private key cannot be serialised into an API response even by mistake. Bad ciphertext raises `RuntimeError` rather than returning empty values. Private keys are never printed by `setup_platform_wallet`. There is no custom application logger beyond `uvicorn`'s access log (method/path/status, no bodies).

CORS is a demo convenience for the local Vite origin, not an authentication mechanism. Production would restrict origins, put keys in a secrets manager, rotate Fernet keys with a dual-decrypt window, and stop using SQLite as the vault for encrypted seeds.

12 Regulatory considerations

This section is an understanding of the problem, not a legal opinion (University of Cape Town, 2026a). A Testnet IOU does not make the operator a licensed money transmitter, and it does not take the corridor outside FICA or SARB rules.

KYC and AML. South African accountable institutions under FICA must identify customers and understand source of funds before providing money-remittance services. Our mock KYC collects the brief's fields, encrypts the ID number, and gates sending and cash-out on admin approval. That is a teaching model of a control, not a CDD programme: we do not screen against sanctions lists, we do not verify documents against Home Affairs, and we do not file suspicious-transaction reports.

Transaction monitoring (not fully automated). A real remittance business is expected to monitor velocity, structuring, mule patterns, and corridor risk. We enforce daily and monthly ZAR caps and keep a full remittance and settlement audit trail, including XRPL hashes. We do *not* ship an AML product: no rules engine, no alert queue, no SAR workflow. Operators would still need people and systems we have not built.

Custody of crypto assets. Recipient wallets are custodial: the platform generates Testnet keys, encrypts them, and signs on the customer's behalf. That is convenient and it is also a custody business. In production that would engage crypto-asset service-provider expectations (safekeeping, segregation, operational resilience) under the evolving South African CASP / FSCA framework. We store keys encrypted with a dedicated Fernet key; we do not offer self-custody withdrawal of the secret.

SARB and exchange control. Cross-border rand payments sit inside South African exchange-control and authorised-dealer arrangements. A stablecoin hop does not create a new legal corridor. Our cash-in is simulated, so we never actually take rand across the border. A live system would still need an authorised channel for the fiat legs and would still need to report what the SARB requires. RLUSD on Testnet is not a SARB-approved mechanism for discharging an exchange-control obligation.

Licensing. A commercial Western Union competitor in South Africa typically needs a FICA accountable-institution posture, Reserve Bank authorisation where the activity is a remittance or payment service, and—if offering crypto custody or exchange—the applicable FSCA crypto-asset authorisation. Agent cash-in/out may also engage payment-system and consumer-protection rules (clear fees, receipts, complaint handling). This coursework prototype is not licensed and must not be pointed at real customer funds or Mainnet credentials (University of Cape Town, 2026a).

Consumer protection and safeguarding follow from the same honesty: quoted fees should match charged fees (our quote locks the breakdown for 15 minutes); customer secrets should not leak (Section 11); and simulated balances must not be described as redeemable bank money. We do not hold client fiat in a trust account because we never collect real fiat. The web UI in Section 15 shows the locked fee breakdown, the MG tracking reference, and an About section (what Baobab is, the course, the team, and the repository). That is coursework disclosure, not a licensed complaints desk.

13 Assumptions and limitations

Assumptions we adopted on purpose: one customer identity for send and receive, so the stored roles are `customer` and `admin` rather than the three roles FR-04 names (a person who sends today may receive tomorrow, and a separate recipient account type would force a second registration); admins only via `scripts/create_admin.py`; one KYC row with overwrite-on-resubmit until approved; beneficiary auto-match rather than invite-to-register; hybrid treasury plus per-recipient XRPL accounts; issuer address and currency code as `.env` so UCTUSD versus RLUSD is a config change (University of Cape Town, 2026b); Redis Streams as the queue; SQLite for development; quote TTL of 900 seconds. The requirements say RLUSD throughout; the prototype settles in UCTUSD, the course's issued token, and keeps the `rlusd_amount` field names so switching to official RLUSD is an environment change. Mobile and identification numbers are checked for format only, and generically (8–15 digits; 6–20 characters), because a beneficiary's country and an applicant's nationality are free text: a South-Africa-only rule (+27, a 13-digit ID with checksum) would reject legitimate foreign recipients and passport holders.

Remaining limitations of *this* fork, not of the old sketch:

- SQLite's single-writer model is not a production remittance ledger.
- Cash-in and fiat cash-out are simulated; complete still burns reserved UCTUSD on-chain by paying the issuer.
- Automated tests mock XRPL (faucet, TrustSet, Payment). Redis is real (DB 15).
- Redis is required at runtime and in pytest.
- Login latency is dominated by bcrypt (see Section 14).
- No AML monitoring product, no sanctions screening, no production key-management service.
- Format validation is not verification: there is no OTP for mobile numbers and no Home Affairs or passport check on identification numbers.
- The seeding and smoke-test scripts raise the verified-tier caps so their own runs are not blocked, and do not restore them. Caps must be reset to R3,000 / R25,000 before a demo; a development database was found still holding the raised caps.
- Daily/monthly limit checks and the quote insert are separate steps with no row lock between them; two concurrent quote requests from the same sender could both read usage under the limit and jointly exceed it. Acceptable for a single-process academic prototype, not for a production ledger.

14 Performance summary

HTTP load, queue enqueue, and five live Testnet Payments were re-run on this fork on 7 September 2026 (50 concurrent synthetic users, 60 s, Windows, single-process uvicorn and SQLite). The tables and charts live in `PERFORMANCE_TESTING.md` and `backend/perf/results/`.

The HTTP mix delivered 2,218 requests, zero failures, about 37 req/s. Authenticated reads sat around 20–33 ms at the median; `POST /remittances` about 66 ms median. `POST /auth/login`

remains the outlier (about 850 ms median) because bcrypt is deliberately expensive. Queue enqueue on this OneDrive-backed SQLite host measured about 29 messages/s (a 1 September Mac sample of the same Redis path was 517 msg/s). Five real Testnet Payments completed 5/5 and averaged about 17.5 s each—ledger close time, not application CPU (a 1 September sample of the same script was 12.4 s/tx). Relative bottlenecks—bcrypt on login, consensus on settlement—are unchanged in kind.

15 Web UI

The marked web application is `frontend/`: Vite, React, TypeScript, and React Router. It calls the FastAPI JSON API with `VITE_API_URL` (default <http://127.0.0.1:8000>). The landing page is a marketing scroll (hero, quote explainer, beneficiary and tracking sections, a seven-question FAQ as `#faq`, an About section as `#about` with the course, team and repository, footer). The platform is branded Baobab, with a baobab-tree logo in the navigation bar and as the browser icon. Login, register, and the authenticated shells use a DASH-style grid-trail background. Landing motion is GSAP ScrollTrigger plus original Three.js objects—not third-party brand assets.

Table 12 is the shipped screen map. The quote screen shows send amount, locked transaction fee, net ZAR after that fee, FX margin, rate, and UCTUSD. Empty states are implemented, not planned: unverified senders see KYC pending, not a send form; expired quotes show expired and hide cash-in; a recipient without a Testnet account yet still sees a zero spendable wallet row; settlement failure shows the worker reason (for example `tecPATH_DRY`) rather than a dummy hash.

Navigation is a sidebar of icon tiles grouped into Account and Admin sections, with the current page highlighted. Two confirmations sit on top of the API’s own checks, which stay authoritative: the send form asks the sender to confirm amount and beneficiary before a quote is locked, and an admin who approves or rejects KYC gets a dialog naming the applicant and the outcome, in addition to the inline banner. The settlement and cash-out admin screens follow the same pattern: running the worker, retrying a message, and approving, completing or failing a cash-out each ask for confirmation first, stating what the action does and whether it can be undone, then report the outcome in a dialog as well as the banner. A completed cash-out’s dialog links its burn hash to the Testnet explorer. Status badges use solid fills with one colour per meaning, and **approved** is deliberately not the colour of **completed** or **settled**, because both appear side by side in the cash-out queue. The beneficiaries screen shows the add form and the saved list side by side, so the Linked or Unlinked badge is visible next to the form. The quote and track screens show a QR code encoding the transfer’s track-page URL (the site origin, or `VITE_PUBLIC_URL` when set); scanning it on a phone opens the track page after sign-in, subject to the same sender, linked-recipient or admin check. A phone cannot open `localhost`, so scanning is tested on the hosted build or with `VITE_PUBLIC_URL` set to the laptop’s network address. The sender home screen shows daily and monthly usage as bars (amber from 80%, red at the limit) beside the remaining allowance. The wallet screen explains that it holds money received in UCTUSD and that sending is paid in rand at cash-in, so a sender who has received nothing sees an empty wallet with a link to start a quote.

Table 12: Shipped screens mapped to routes and endpoints.

Screen	Route	Endpoints
Landing / FAQ / About	/, #faq, #about	static; About states the course, team and GitHub
Register / login	/register, /login	POST /auth/register, /login, /logout
Sender home	/app	GET /users/me, /kyc/me/status, /limits/me
Profile	/app/profile	GET/PATCH /users/me
KYC form	/app/kyc	POST /kyc, GET /kyc/me
Beneficiaries	/app/beneficiaries	GET/POST /beneficiaries
New quote / quote	/app/send, /app/send/{id}	POST /remittances, .../cancel, .../cash-in
History / track	/app/history, /app/track/{ref}	GET /remittances/me, /remittances/track/{ref}
Recipient wallet	/app/wallet	GET /wallet/me
Cash-out	/app/cash-out	POST /cash-outs, GET /cash-outs/me
Admin KYC	/admin/kyc	GET /kyc, POST .../approve reject
Admin cash-in	/admin/cash-in	GET /remittances, POST .../confirm-cash-in
Admin settlement	/admin/settlement	GET /admin/settlement, POST .../run, .../{id}/retry
Admin cash-out	/admin/cash-out	GET /cash-outs, POST .../approve complete fail
Admin config	/admin/config	GET/PUT /admin/fee-config, /admin/limit-tiers
Platform wallet	/admin/platform-wallet	GET /admin/platform-wallet; TrustLine helper

Admins are created with `scripts/create_admin.py`, never via public register. The examiner demo is the UI at <http://127.0.0.1:5173/> (or 5174) against the API on 8000: register or sign in, KYC, beneficiary, quote, cash-in, admin confirm, settlement run or retry, track, then recipient wallet and optional cash-out. Swagger at `/docs` remains a developer tool, not the marked product.

References

- FinMark Trust (2024). South africa to the rest of SADC remittances market assessment 2024. Technical report, FinMark Trust.
- International Monetary Fund (2025). South africa: Technical assistance report—cross-border payments: South africa–zimbabwe corridor diagnostic. Technical Report 25/93, International Monetary Fund.
- Rateweb (2026). Your transfer fee ignores where you’re sending. that costs you. Online article analysing World Bank Remittance Prices Worldwide 2025 Q1 figures for South African out-bound corridors. Secondary citation of World Bank RPW 2025 Q1. Reports Western Union and MoneyGram flat fees of about R214–R217 on a surveyed R1,370 send.
- University of Cape Town (2026a). ECO5040W class project brief: XRPL-based FX remittance platform using RLUSD. Course brief, Financial Software Engineering.
- University of Cape Town (2026b). ECO5040W project resources. Trust lines, IOUs, RLUSD vs fallback IOU, recommended stack.
- World Bank (2025). Remittance prices worldwide. Technical report, World Bank. Quarterly survey of the cost of sending remittances. Primary source for corridor fee comparisons.